Laboratorio 5

ICC-708

Alumnos:
Luis Bravo.
Juan Inzunza.
Carrera: Ingeniería Informática
Fecha: 25-03-2026

1. Descripción de la estructura implementada

Se implementó una tabla hash con encadenamiento en Rust. La estructura consiste en un arreglo de 211 cubetas, donde cada cubeta es un vector dinámico de pares (String, usize). Cuando dos llaves distintas producen el mismo índice, ambas coexisten en la misma cubeta dentro de ese vector, sino se añade al vector general.

Se implementaron dos funciones hash para comparar su comportamiento, una que calcula mediante sumas (acumulando los valores ASCII de cada carácter) y otra polinomial (acumula trabajando con base 31 para calcular). La tabla en sí soporta inserción, búsqueda y eliminación- además registra métricas de colisiones, cubetas ocupadas, tamaño máximo de cubeta y factor de carga.

Se ocupó Rust por que “no debería importar el lenguaje”- y por ende se ocuparon cosas idiomáticas de este lenguaje (como enums, muts, match, pattern matching, etc). El programa ocupó los crates de Time, Rand y cmp.

Link repositorio: https://github.com/JJIRivas/Lab-prog-avanzada/tree/Lab5---Hashes

---

2. Tabla comparativa

Dataset	Función	Colisiones	Cubetas Ocupadas	Max Cubeta	Tiempo (s)
Aleatorio	Suma	791	209	12	0.000147
Aleatorio	Polinomial	790	210	10	0.000176
Secuencial	Suma	945	55	70	0.000279
Secuencial	Polinomial	789	211	8	0.000138
Agrupado	Suma	945	55	70	0.000268
Agrupado	Polinomial	789	211	10	0.000119

---

3. Discusión

Con datos aleatorios ambas funciones se comportan de manera similar pero se empieza a notar una diferencia a la hora de trabajar con datos secuenciales y agrupados. La función Suma se empieza a ver afectada más que nada por llaves como "user0", "user1", "user2" que tienen valores ASCII muy similares. La variación es mínima y todas tienden a caer en los mismos índices. Con "aaa0", "aaa1", etc. el problema es aún más severo.

En contra, la función Polinomial maneja estos casos con consistencia porque al ocupar bases numéricas hace que el peso de cada carácter en el cálculo hace que llaves similares produzcan índices distintos.

Es de notar que la diferencia de velocidad observada no proviene del algoritmo en sí, sino de la distribución.

---

4. Conclusión técnica

La elección de la función hash tiene un impacto significativo en el rendimiento de una tabla hash, especialmente según el tipo de datos que se maneje. La función de Suma aunque más simple de aplicar, es sensible a patrones en las llaves, ya que no considera el orden de los caracteres, llevando el tiempo de O(1) hacia O(n) relativamente. En cambio, la función Polinomial al incorporar la posición de cada carácter mediante una base, genera índices más dispersos y mantiene una distribución uniforme independientemente del patrón de los datos. Esto se traduce en menos colisiones, cubetas más balanceadas, tiempos de inserción menores, etc.

Es importante considerar que para aplicaciones donde el tipo de dato es variable/desconocido, una función hash que considere la estructura interna más que el algoritmo en sí es “clave”.